

APEX-M · Local Setup Package

Developer handoff · run APEX-M against in-process mocks · Independence-clean by construction

Self-contained handoff for spinning up the **APEX-M (Microsoft variant)** codebase on a developer laptop. Two install paths included:

1. **Canonical** — `uv` workspace install (matches what the repo actually uses)
2. **Pip fallback** — `requirements.txt`-based install for environments without `uv`

Both land in the same place: APEX-M agent code running against **in-process mock implementations** of every Microsoft SDK call. No Azure subscription required. No client data touched. Independence-clean by construction.

What's in this package

File	Purpose
README.md	This file — quickstart + caveats
requirements.txt	Pip-equivalent dependency list, derived from <code>apex-m/pyproject.toml</code>
.env.example	Every env var an APEX-M agent container reads on the laptop substrate
pyproject-snippet.toml	Original dependency declaration block from <code>apex-m/pyproject.toml</code> (for <code>uv</code> users)
run-local-mock.sh	Bash quickstart — installs deps + boots one agent against mocks
run-local-mock.ps1	PowerShell equivalent for Windows

Prerequisites

Requirement	Why
Python ≥ 3.12	Hard requirement from <code>apex-m/pyproject.toml</code> (<code>requires-python = ">=3.12"</code>)
git	To clone the APEX repo
uv (<i>recommended</i>)	Native package manager the repo uses. Install: <code>pip install uv</code> or <code>brew install uv</code>
Docker Desktop (<i>optional</i>)	Only needed for the full docker-compose path with mock-container substrate. Pure in-process mocks work without Docker.

Path A · Canonical install (uv workspace)

```
# 1) Clone the APEX repo (skip if you already have it)
git clone https://github.com/Deloitte-US-Consulting/APEX.git
cd APEX

# 2) Sync the whole workspace, including apex-m runtime + dev extras
uv sync --extra runtime --extra dev

# 3) Copy the env template into the apex-m runtime folder
cp docs/guides/APEX-M-Local-Setup/.env.example apex-m/.env

# 4) Run agent tests against mocks (no Azure tenant required)
cd apex-m
uv run pytest -m "not integration"
```

uv sync resolves the apex-core workspace dependency from packages/apex-core/ automatically — no manual editable install needed.

Path B · Pip fallback

For environments where uv is unavailable (locked-down corp builds, air-gapped CI lanes, etc.):

```
# 1) Create a Python 3.12+ venv
python -m venv .venv
# Activate (Windows PowerShell)
.venv\Scripts\Activate.ps1
# Activate (Bash/macOS/Linux)
source .venv/bin/activate

# 2) Install the workspace-local apex-core in editable mode FIRST
pip install -e packages/apex-core

# 3) Then install apex-m with runtime + dev extras
pip install -e "apex-m[runtime,dev]"
# ...or use the flat list:
pip install -r docs/guides/APEX-M-Local-Setup/requirements.txt

# 4) Verify
python -c "import apex_m; print('apex-m imported OK')"
```

Configure the .env

Copy .env.example to apex-m/.env (or ~/apex-deployments//.env if you're following the wizard-rendered layout) and adjust:

```
# Minimal edits to make the mocks usable
APEX_TENANT=your-engagement-slug
APEX_USE_CASE_ID=rc-e2e-03--default      # or any other catalog ID
APEX_PERSONAS=marisol-reyes-store-ops,daniel-chen-merch-director
APEX_HITL_MARKDOWN_PCT=30               # tune to your scenario

# Leave the mock endpoints alone – they're auto-set when docker-compose
# is involved, and ignored when running pure in-process mocks
```

Critical knob: APEX_FORCE MOCK=true routes every Microsoft SDK call to a Mock* implementation. Keep this set on the laptop substrate. Flipping to false requires a real Microsoft tenant and an Azure subscription with appropriate role assignments.

Run one agent (in-process, no Docker)

```
# After Path A or B install + env config
cd apex-m
APEX_SUBSTRATE=laptop APEX_FORCE MOCK=true \
  uv run python -m apex_m.cli run-agent \
    --service-code RC-E2E-03 \
    --scenario cold-chain-pricing
```

Or use the included quickstart scripts:

```
# Bash / macOS / Linux / WSL
./run-local-mock.sh
```

```
# Windows PowerShell
.\run-local-mock.ps1
```

Run the full docker-compose stack (optional)

If you want the full substrate experience (mock Foundry/Fabric/Purview/Redis containers + agent containers):

```
mkdir -p ~/apex-deployments/my-engagement
cp .env apex-m/infra/docker-compose/base.docker-compose.yml ~/apex-deployments/my-engagement/
cd ~/apex-deployments/my-engagement
docker-compose up
docker-compose logs -f apex-m-rc-e2e-03-cold-chain-pricing
```

Known caveat — Sprint 47 work-in-progress

From `apex-m/infra/docker-compose/README.md`:

Sprint 47 — *Real mock service images. Currently `ghcr.io/apex/mock-foundry:0.1.0` etc. don't exist; first client engagement Lab work builds them.*

Until those images publish, `docker-compose up` will fail at the image-pull step. Workaround: use the in-process mocks (`APEX_FORCE MOCK=true` + direct Python entry) per the script above. The exact same Mock* classes satisfy the same APEX-Core protocols — the only difference is they live in your Python process instead of HTTP shim containers.

What you get vs. what's deferred

Capability	Laptop substrate	Dev substrate (Azure Lab)	Prod substrate
Agent reasoning loop	[YES] Full	[YES] Full	[YES] Full
Mock Microsoft SDKs	[YES] In-process	[NO] Real Azure	[NO] Real Azure
HITL gates	[YES] Synthetic	[YES] Synthetic	[YES] Real personas
Audit-row emission	[YES] stdout	[YES] Lab Purview	[YES] Prod Purview
Sentinel detections	[NO] N/A	[YES] Lab workspace	[YES] Prod workspace

Capability	Laptop substrate	Dev substrate (Azure Lab)	Prod substrate
Security Copilot	[NO] N/A	[YES] 1 SCU	[YES] 4 SCU
Independence posture	[OK] Clean by construction	[CAUTION] Lab tenant only	[OK] Per engagement contract

Source: Deployment Guide §2 — Substrate-Aware Architecture.

Troubleshooting

Symptom	Likely cause	Fix
ModuleNotFoundError: apex_core	Workspace dep not editable-installed	<code>pip install -e packages/apex-core</code> first
ImportError: agent_framework	Runtime extras not installed	<code>uv sync --extra runtime</code> or <code>pip install "apex-m[runtime]"</code>
<code>docker-compose pull</code> fails on <code>ghcr.io/apex/mock-*</code>	Sprint 47 images haven't shipped	Use in-process mocks; skip <code>docker-compose</code> for now
Tests fail with auth errors	<code>.env</code> missing or <code>APEX_FORCE MOCK=false</code>	Ensure <code>APEX_FORCE MOCK=true</code> for laptop work
<code>azure-ai-projects</code> resolves to wrong version	Beta cadence churns fast	Pin to <code>1.0.0b3</code> per <code>pyproject.toml</code> ; do not upgrade ad hoc

References (inside the APEX repo)

- `apex-m/pyproject.toml` — canonical dependency declaration
- `apex-m/README.md` — Microsoft platform coverage matrix
- `apex-m/infra/docker-compose/README.md` — operator workflow
- `apex-m/infra/docker-compose/.env.example` — source of `.env.example` in this package
- `docs/book/Professional-APEX-M-Deployment-Guide.html` — Ch 2 Substrate Architecture, Ch 6B Sentinel + Security Copilot
- `docs/apex-core/Independence-Posture.md` — why laptop substrate is Independence-clean

Package version: 1.0 · Generated: 2026-05-29 · For APEX-M version: 0.1.0